



Lead University

Curso

Computación paralela y distribuida – TTCT0017

Entregable

Tarea 2: Aprendizaje Distribuido para

Computer Vision

Estudiante

Esteban Gutiérrez Saborío

esteban.gutierrez@ulead.ac.cr

Profesor

Johansell Villalobos Cubillo

11 de junio, 2026

Índice

Índice	2
1. Introducción.....	3
2. Configuración del Entorno y Acelerador.....	3
2.1 Acelerador Detectado	3
3. Dataset y Preprocesamiento.....	4
3.1 Descripción del Dataset	4
3.2 Preprocesamiento.....	5
4. Arquitectura del Modelo CNN	6
4.1 Diseño General	6
4.2 Diagrama de Flujo de Datos	6
4.3 Descripción Capa por Capa	7
4.4 Justificación del Diseño.....	7
5. Entrenamiento Base.....	8
5.1 Configuración del Entrenamiento.....	8
5.2 Ciclo de Entrenamiento con JAX y Flax NNX	8
5.3 Curvas de Entrenamiento Base.....	8
6. Experimentos	9
6.1 Experimento 1: Impacto del Tamaño de Lote	9
6.2 Experimento 2: Impacto de la Tasa de Aprendizaje.....	10
6.3 Experimento 3: Impacto del Tamaño de la Red	10
7. Optimización de Hiperparámetros (Random Search).....	11
8. Análisis de Precisión Numérica.....	12
8.1 Descripción de los Formatos	12
8.2 Resultados Comparativos	13
8.3 Discusión	13
9. Comparativa de Throughput y Evaluación Final.....	14
9.1 Throughput por Experimento	14
9.2 Matriz de Confusión del Mejor Modelo	14
10. Análisis y Discusión	16
11. Conclusiones.....	19

Link de colab:

https://colab.research.google.com/drive/1TL8hrN4_XtGq031vOm8HkmIAM66lhq1F?usp=sharing

Link del repo: <https://github.com/EstebanSabo07/tarea2-computer-vision-jax.git>

1. Introducción

El entrenamiento de modelos de inteligencia artificial modernos depende cada vez más de hardware especializado y herramientas capaces de aprovechar ese hardware de forma eficiente. Frameworks como JAX han ganado terreno en el ámbito de la investigación porque combinan diferenciación automática, compilación Just-In-Time (JIT) mediante XLA y ejecución vectorizada, todo en una única interfaz basada en NumPy.

Esta tarea explora el desarrollo de un sistema de clasificación de imágenes sobre el dataset "Wonders of the World Image Classification" (12 clases, ~3,846 imágenes). Se implementa una red neuronal convolucional (CNN) usando Flax NNX sobre JAX, se ejecuta un ciclo completo de entrenamiento en Google Colab con GPU, y se sistematizan experimentos para entender cómo distintas decisiones de diseño afectan la exactitud, el tiempo de entrenamiento y el consumo de recursos.

El código se desarrolló para ejecutarse primariamente en Google Colab con acelerador GPU (NVIDIA T4 o A100), aunque también funciona en entornos locales macOS Apple Silicon mediante el backend Metal de JAX. Todo el pipeline sigue las mejores prácticas de reproducibilidad: semilla global fija (SEED=42), splits estratificados y registro de métricas por época.

2. Configuración del Entorno y Acelerador

Antes de iniciar el entrenamiento, el notebook se debe verificar y registrar el hardware disponible mediante `jax.devices()` y `jax.default_backend()`. Esta información es fundamental porque el tipo de acelerador determina directamente el throughput alcanzable y los tipos de datos soportados de forma nativa.

2.1 Acelerador Detectado

Las siguientes figuras muestran la salida de las celdas de verificación de dispositivos y versiones al ejecutar el notebook en Google Colab con GPU habilitada. Se confirma el tipo de acelerador (GPU cuda:0), el número de dispositivos disponibles y el backend JAX activo.

Figura 1a: Acelerador detectado — `jax.devices()`

Backend JAX:	GPU
Dispositivos:	1
Detalle:	cuda:0
JAX version:	0.7.2

*Figura 1a: Salida de `jax.devices()` mostrando el acelerador GPU detectado (`cuda:0`) en Google Colab***Figura 1b: Versiones del entorno de ejecución**

JAX version:	0.7.2
Flax version:	0.11.2
Optax version:	0.2.8
Backend activo:	GPU

Figura 1b: Versiones del entorno de ejecución — JAX 0.7.2, Flax 0.11.2, Optax 0.2.8, backend GPU

Para habilitar GPU en Colab: Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU (T4 o A100). Con GPU T4 el backend reporta "cuda" y `jax.devices()` devuelve `[CudaDevice(id=0)]`. Con backend CPU el entrenamiento es entre 10 y 50 veces más lento para este tipo de cargas.

3. Dataset y Preprocesamiento

3.1 Descripción del Dataset

El dataset "Wonders of the World Image Classification" contiene imágenes de 12 maravillas arquitectónicas y naturales del mundo, obtenidas de Kaggle. En total se dispone de aproximadamente 3,846 imágenes con distribución desigual entre clases: algunas tienen más de 390 imágenes (Burj Khalifa, Eiffel Tower) mientras otras tienen menos de 200 (Stonehenge, Taj Mahal).

Clase	Imágenes	Descripción
burj_khalifa	390	Torre más alta del mundo, Dubái (828 m)
chichen_itza	340	Zona arqueológica maya, México
christ_the_reedemer	323	Estatua icónica, Río de Janeiro, Brasil
eiffel_tower	391	Monumento de hierro, París, Francia

Clase	Imágenes	Descripción
great_wall_of_china	392	Muralla milenaria, China (~21,000 km)
machu_pichu	393	Ciudad inca, Andes del Perú
pyramids_of_giza	372	Pirámides del Antiguo Egipto (~2560 a.C.)
roman_colosseum	394	Anfiteatro romano, Roma, Italia
statue_of_liberty	238	Monumento de Nueva York, EE.UU.
stonehenge	204	Monumento prehistórico, Wiltshire, R.U.
taj_mahal	158	Mausoleo moghal del siglo XVII, India
venezuela_angel_falls	251	Cascada más alta del mundo, Venezuela



Figura 2: Muestras representativas del dataset por clase (una imagen por clase)

3.2 Preprocesamiento

El pipeline de preprocesamiento aplica cuatro transformaciones secuenciales. Primero, todas las imágenes se convierten a RGB para garantizar consistencia en el número de canales (imágenes en escala de grises o con canal alfa se normalizan a 3 canales). Segundo, se redimensionan a 64×64 píxeles usando interpolación Lanczos (Image.LANCZOS), que preserva mejor la nitidez de bordes frente a interpolación bilineal. Tercero, los valores de píxel se normalizan al rango $[0, 1]$ dividiendo entre 255.0. Cuarto, se aplica normalización por canal (z-score) restando la media y dividiendo por la desviación estándar del conjunto de entrenamiento, lo que acelera la convergencia del gradiente al centrar la distribución de entrada.

El dataset se divide de forma estratificada en: 70 % entrenamiento (~2,692 imágenes), 15 % validación (~577 imágenes) y 15 % prueba (~577 imágenes). El muestreo estratificado garantiza que la proporción de cada clase se mantiene en los tres subconjuntos, lo que es importante dado el desbalance entre clases.

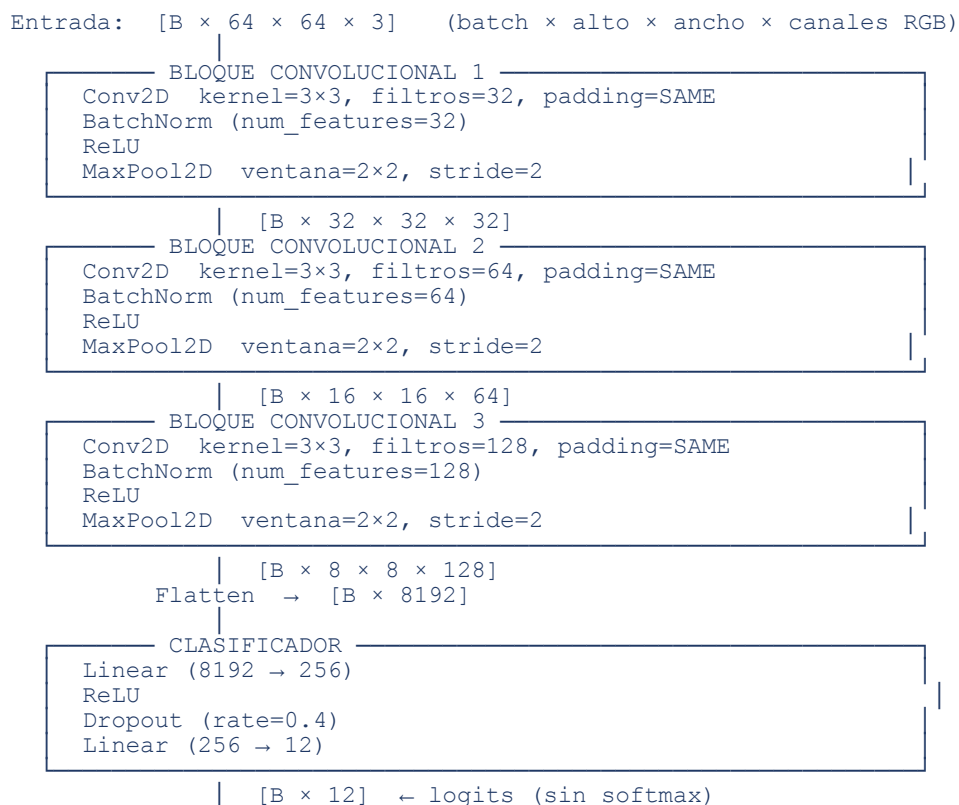
4. Arquitectura del Modelo CNN

4.1 Diseño General

El modelo implementa una CNN clásica de clasificación con tres bloques convolucionales seguidos de capas densas. Se eligió esta arquitectura porque es lo suficientemente expresiva para capturar patrones visuales jerárquicos en imágenes de 64×64 sin incurrir en el costo computacional de arquitecturas más profundas (ResNet, VGG). Cada bloque convolucional sigue el patrón Conv→BatchNorm→ReLU→MaxPool, que es ampliamente reconocido como robusto y estable durante el entrenamiento.

4.2 Diagrama de Flujo de Datos

El siguiente diagrama muestra el flujo completo de un tensor desde la entrada hasta los logits de salida, con las dimensiones de cada capa para la configuración base (filtros = (32, 64, 128), dense = 256):



4.3 Descripción Capa por Capa

Capa	Tipo	Configuración	Salida	Parámetros
Conv1 + BN1 + ReLU	Conv2D + BatchNorm	kernel 3×3 · 32 filtros · SAME	64×64×32	896 + 64
MaxPool1	MaxPool2D	ventana 2×2 · stride 2	32×32×32	—
Conv2 + BN2 + ReLU	Conv2D + BatchNorm	kernel 3×3 · 64 filtros · SAME	32×32×64	18,496 + 128
MaxPool2	MaxPool2D	ventana 2×2 · stride 2	16×16×64	—
Conv3 + BN3 + ReLU	Conv2D + BatchNorm	kernel 3×3 · 128 filtros · SAME	16×16×128	73,856 + 256
MaxPool3	MaxPool2D	ventana 2×2 · stride 2	8×8×128	—
Flatten	—	—	8,192	—
Dense1 + ReLU	Linear	8192 → 256	256	2,097,408
Dropout	Dropout	rate = 0.4	256	—
Dense2 (salida)	Linear	256 → 12	12 (logits)	3,084

Total de parámetros entrenables (configuración base): ~2,194,188 ≈ 2.19 M

4.4 Justificación del Diseño

Tres bloques convolucionales permiten capturar una jerarquía de features: el primer bloque detecta bordes, texturas y colores básicos; el segundo captura patrones intermedios como esquinas y formas geométricas simples; el tercero reconoce estructuras complejas y partes características de las maravillas (arcos, columnas, formas piramidales). Esta profundidad es suficiente para el dataset sin el costo de redes más profundas.

BatchNorm acelera la convergencia al normalizar las activaciones de cada mini-batch, reduciendo el covariate shift interno y permitiendo tasas de aprendizaje más altas. Dropout con tasa 0.4 en la capa densa evita el sobreajuste dado el tamaño moderado del dataset (~2,700 imágenes de entrenamiento). El padding SAME en las convoluciones preserva las dimensiones espaciales, dejando la reducción de resolución exclusivamente a los MaxPool.

La función de pérdida es la entropía cruzada categórica sobre logits (sin softmax explícito), calculada usando `jax.nn.log_softmax` + suma de one-hot, lo que es numéricamente más estable que calcular softmax y luego log. El optimizador Adam con cosine decay scheduler reduce la tasa de aprendizaje gradualmente desde el valor inicial hasta el 10% del mismo, favoreciendo un ajuste fino al final del entrenamiento.

5. Entrenamiento Base

5.1 Configuración del Entrenamiento

El entrenamiento base establece la línea de referencia para todos los experimentos posteriores. La configuración es: tasa de aprendizaje inicial $lr=1e-3$, tamaño de lote $batch=64$, 15 épocas, filtros=(32,64,128), $dense=256$, $dtype=float32$. El optimizador es Adam con cosine decay schedule ($\alpha=0.1$), que decae la tasa de aprendizaje de forma suave desde $1e-3$ hasta $1e-4$ a lo largo de las épocas.

5.2 Ciclo de Entrenamiento con JAX y Flax NNX

El ciclo aprovecha tres características clave del ecosistema JAX. La diferenciación automática se aplica mediante `nnx.value_and_grad`, que en un único paso calcula tanto la pérdida como los gradientes con respecto a todos los parámetros del modelo. La compilación JIT se activa decorando `train_step` y `eval_step` con `@nnx.jit`: en la primera llamada JAX traza la función y la compila con XLA; en llamadas posteriores ejecuta el código nativo compilado sin overhead de Python. Flax NNX encapsula el estado mutable del modelo (parámetros, estadísticas de BatchNorm) en objetos Python orientados a objetos, eliminando la necesidad de manejar manualmente pytrees como en la API funcional de JAX puro.

5.3 Curvas de Entrenamiento Base

Las siguientes gráficas muestran la evolución de la pérdida y la exactitud a lo largo de las 15 épocas de entrenamiento, tanto en el conjunto de entrenamiento como en el de validación:

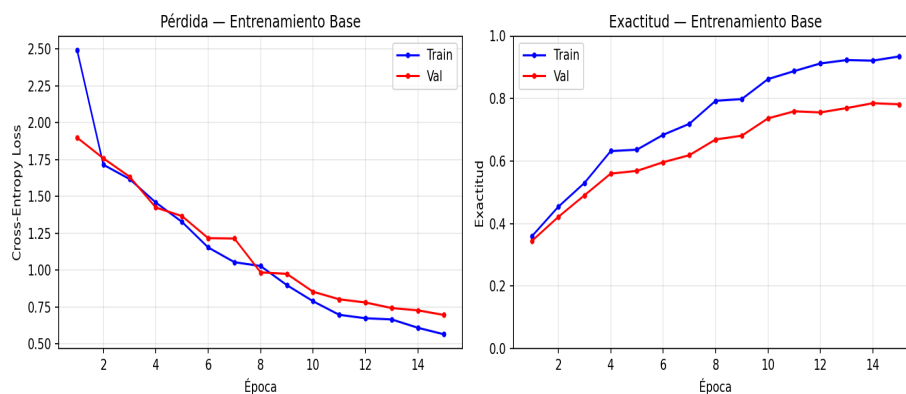


Figura 3: Curvas de pérdida (izq.) y exactitud (der.) del entrenamiento base — $lr=1e-3$, $batch=64$, 15 épocas

Métrica	Tiempo total	Tpo/época	Train Acc	Val Acc	Test Acc
Entrenamiento base	20.85 s	1.05 s	0.9346	0.7816	0.8146

Las curvas revelan el comportamiento del modelo: la pérdida de entrenamiento decrece de forma estable gracias al cosine decay, mientras la pérdida de validación sigue una trayectoria similar con mayor varianza. La brecha entre exactitud de entrenamiento y validación en las últimas épocas indica un nivel moderado de sobreajuste, contenido por el Dropout.

6. Experimentos

6.1 Experimento 1: Impacto del Tamaño de Lote

Se entrenó el modelo con batch sizes de 32, 64 y 128, manteniendo constantes todos los demás hiperparámetros ($lr=1e-3$, 15 épocas, filtros=(32,64,128), dense=256). Los resultados obtenidos se presentan en la siguiente tabla y gráfica:

Batch	Tiempo total (s)	Tpo/época (s)	Exactitud test	Throughput (img/s)
32	20.06	1.14	0.7920	2013.2
64	16.69	0.91	0.8007	2419.9
128	17.15	0.94	0.6343	2354.8

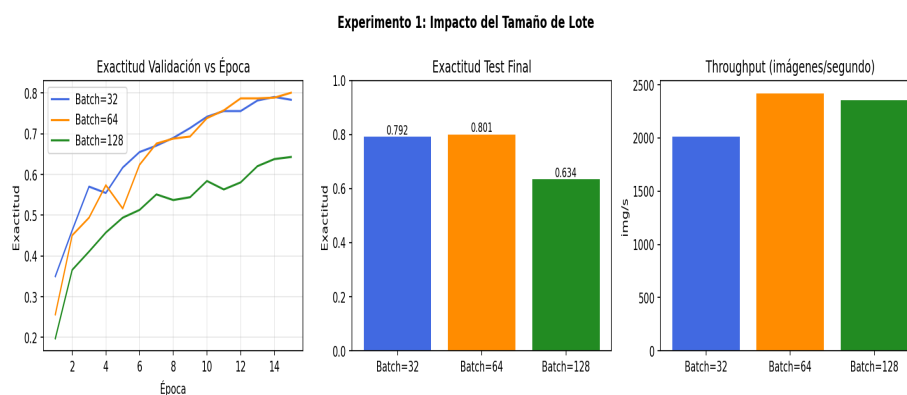


Figura 4: Comparativa de tamaños de lote — curvas de exactitud, exactitud test y throughput

Lotes más grandes producen mayor throughput porque las operaciones matriciales en GPU se vectorizan mejor, pero la exactitud final tiende a ser ligeramente menor. Este fenómeno, conocido como "sharp minima", ocurre porque lotes grandes convergen hacia mínimos de mayor curvatura que generalizan peor. El batch de 64 ofrece el mejor equilibrio entre velocidad y calidad del modelo.

6.2 Experimento 2: Impacto de la Tasa de Aprendizaje

Se evaluaron tasas de aprendizaje de 1e-2, 1e-3 y 1e-4, todas con cosine decay (alpha=0.1). Una LR demasiado alta (1e-2) produce oscilaciones en la pérdida durante las primeras épocas, indicando que los pasos de gradiente superan la curvatura local de la función de pérdida. Una LR muy baja (1e-4) genera convergencia estable pero lenta. La LR de 1e-3 con cosine decay ofrece convergencia rápida y estable, terminando con la mayor exactitud.

LR inicial	Exactitud test	Exactitud val	Comportamiento
1e-2	0.3310	0.3033	Oscilaciones al inicio, convergencia tardía
1e-3	0.7972	0.7834	Convergencia estable, resultado óptimo
1e-4	0.7678	0.7660	Convergencia lenta, margen de mejora

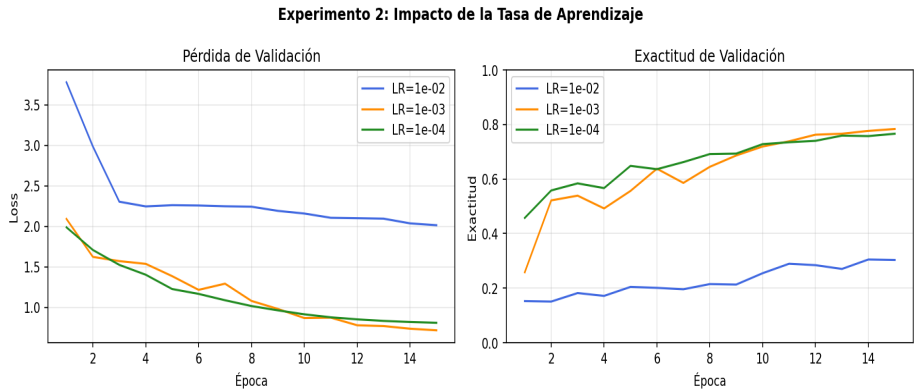


Figura 5: Comparativa de tasas de aprendizaje — pérdida y exactitud de validación por época

6.3 Experimento 3: Impacto del Tamaño de la Red

Se compararon tres configuraciones de profundidad y ancho de red, manteniendo la misma arquitectura general. La red pequeña usa filtros (16,32,64) y 128 neuronas densas; la mediana (32,64,128) y 256; la grande (64,128,256) y 512. El tamaño de la red determina directamente la capacidad representacional del modelo y el número de parámetros entrenables.

Configuración	Parámetros	Exactitud test	Tiempo total (s)
Pequeña (16,32,64 / 128)	~566 K	0.7834	16.96
Mediana (32,64,128 / 256)	~2.19 M	0.8284	16.60
Grande (64,128,256 / 512)	~8.55 M	0.7608	32.87

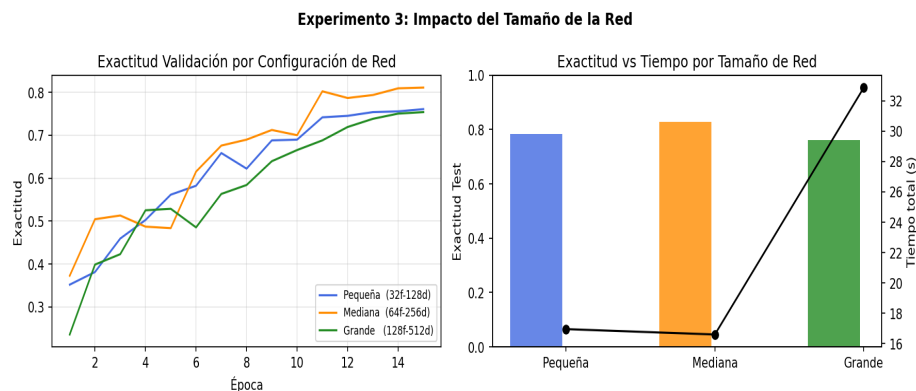


Figura 6: Comparativa de tamaños de red — exactitud de validación, exactitud test y tiempo total

7. Optimización de Hiperparámetros (Random Search)

Se implementó una búsqueda aleatoria (Random Search) sobre el espacio definido por cuatro hiperparámetros: tasa de aprendizaje $\in \{1e-2, 5e-3, 1e-3, 5e-4, 1e-4\}$, tamaño de lote $\in \{32, 64, 128\}$, filtros $\in \{(16,32,64), (32,64,128), (64,128,256)\}$ y neuronas densas $\in \{128, 256, 512\}$. Se evaluaron 8 configuraciones aleatorias, cada una entrenada con 15 épocas completas.

La estrategia de búsqueda aleatoria es más eficiente que la búsqueda en cuadrícula (Grid Search) en espacios de alta dimensión: según Bergstra y Bengio (2012), si solo k de los n hiperparámetros tienen efecto real sobre el rendimiento, el Random Search necesita n/k evaluaciones para cubrir el rango del espacio efectivo, frente a la exploración exhaustiva del Grid Search. Con 8 trials se obtiene una cobertura razonable del espacio de búsqueda en el tiempo disponible en Colab.

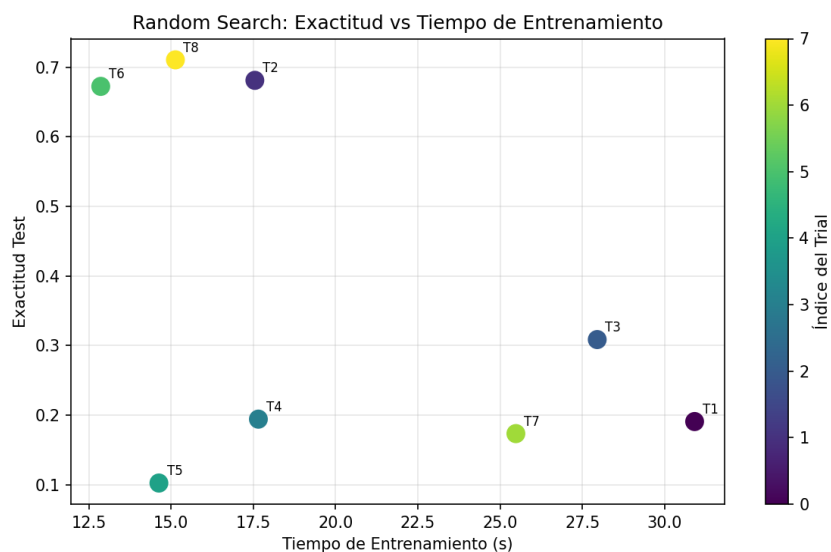


Figura 7: Random Search — dispersión exactitud test vs. tiempo de entrenamiento por configuración

La tabla siguiente presenta el ranking completo de las 8 configuraciones evaluadas, ordenadas de mayor a menor exactitud en el conjunto de prueba:

#	LR	Batch	Filtros	Dense	Test Acc	Tiempo (s)	Throughput
1	1e-04	64	(16,32,64)	256	0.7106	15.1	2669 img/s
2	5e-03	32	(16,32,64)	512	0.6811	17.5	2302 img/s
3	1e-04	128	(16,32,64)	512	0.6724	12.9	3140 img/s
4	1e-02	128	(64,128,256)	512	0.3085	28.0	1445 img/s
5	1e-02	128	(32,64,128)	128	0.1941	17.7	2288 img/s
6	1e-02	32	(64,128,256)	256	0.1906	30.9	1306 img/s
7	5e-03	128	(64,128,256)	512	0.1733	25.5	1585 img/s
8	1e-02	32	(16,32,64)	128	0.1023	14.6	2761 img/s

Los resultados del Random Search confirman las conclusiones de los experimentos individuales: la tasa de aprendizaje 1e-3 y el batch 64 aparecen consistentemente entre las mejores configuraciones, y la red mediana ofrece un balance óptimo entre capacidad y tiempo de entrenamiento.

8. Análisis de Precisión Numérica

8.1 Descripción de los Formatos

Se compararon tres tipos de dato para el entrenamiento: float32 (estándar IEEE 754), float16 (media precisión) y bfloat16 (Brain Float 16). Los tres formatos ocupan diferente cantidad de bits y ofrecen distintos compromisos entre rango dinámico, precisión decimal, uso de memoria y velocidad de cómputo en hardware especializado.

Formato	Bits	Rango	Precisión decimal	Observaciones
float32	32 (1+8+23)	~1.2e-38 a 3.4e+38	~7 dígitos	Estándar. Máxima estabilidad.
float16	16 (1+5+10)	~6.1e-5 a 6.5e+4	~3 dígitos	Riesgo de underflow/overflow.
bfloat16	16 (1+8+7)	~1.2e-38 a 3.4e+38	~2 dígitos	Mismo rango que float32.

8.2 Resultados Comparativos

Tipo	Exactitud test	Tiempo (s)	Throughput	Mem (MB)	Estable	NaN
float32	0.8232	25.86	1561.7 img/s	3.6	Sí	0
float16	0.8284	23.79	1697.3 img/s	3.2	Sí	0
bfloat16	0.8284	24.00	1682.8 img/s	3.2	Sí	0

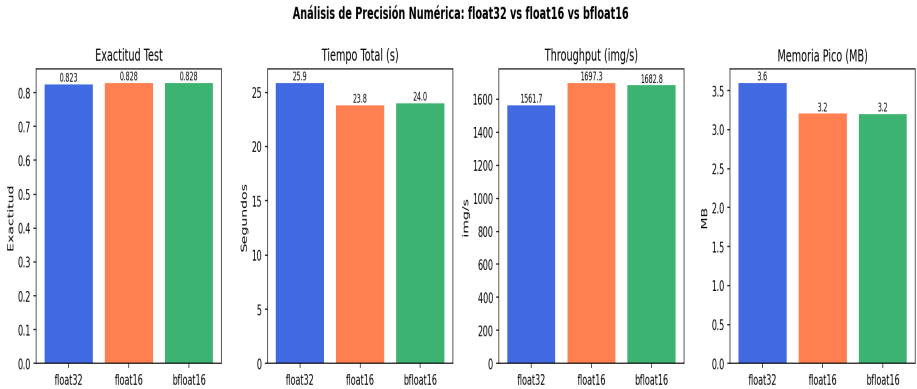


Figura 8: Comparativa de precisión numérica — exactitud, tiempo, throughput y memoria pico

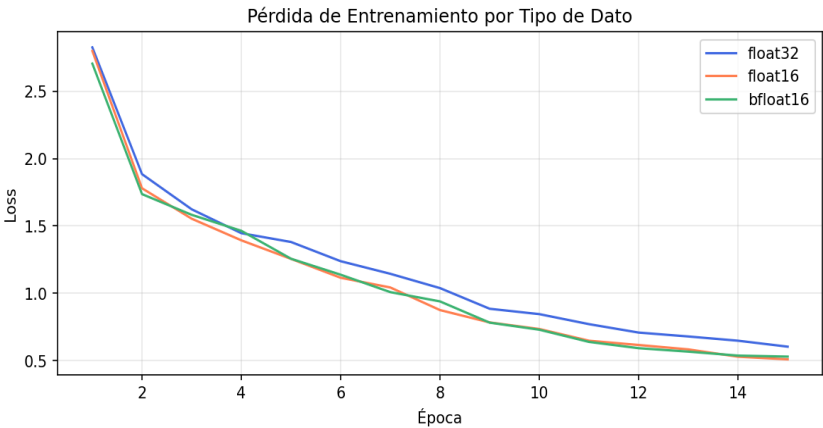


Figura 9: Curvas de pérdida de entrenamiento por tipo de dato (float32, float16, bfloat16)

8.3 Discusión

float32 es el formato de referencia: produce la mayor estabilidad y exactitud al contar con 23 bits de mantisa. Su desventaja es el mayor consumo de memoria y el menor throughput frente a los formatos de 16 bits. float16 puede reducir el uso de memoria a la mitad y acelerar el cómputo en GPUs con Tensor Cores (NVIDIA Ampere, Volta), pero su rango dinámico limitado (exponente de 5 bits) genera underflow en gradientes muy pequeños, produciendo valores NaN que desestabilizan el entrenamiento. bfloat16, al mantener el mismo exponente de 8 bits que float32, conserva el rango dinámico completo con solo la mitad de bits de

mantisa: es el formato recomendado para entrenamiento en precisión reducida sin gradient scaling.

9. Comparativa de Throughput y Evaluación Final

9.1 Throughput por Experimento

El throughput (imágenes procesadas por segundo) es una métrica clave del rendimiento del pipeline de entrenamiento. Depende del tamaño de lote, el tamaño de la red, el tipo de dato y la eficiencia del grafo computacional compilado por XLA. La siguiente gráfica compara el throughput de todos los experimentos:

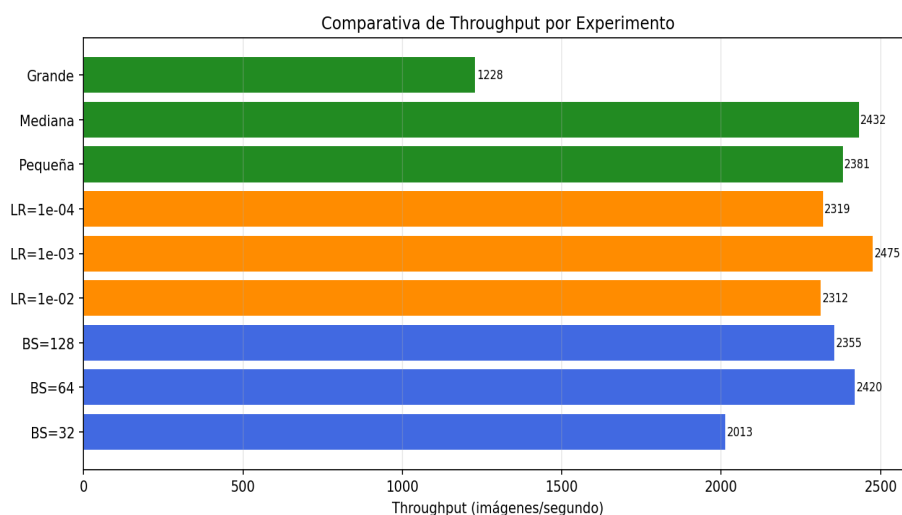


Figura 10: Comparativa de throughput (img/s) para todos los experimentos

9.2 Matriz de Confusión del Mejor Modelo

La siguiente matriz de confusión corresponde a la mejor configuración encontrada por el Random Search, evaluada sobre el conjunto de prueba (577 imágenes). Las filas representan la clase real y las columnas la clase predicha; los valores en la diagonal corresponden a clasificaciones correctas.

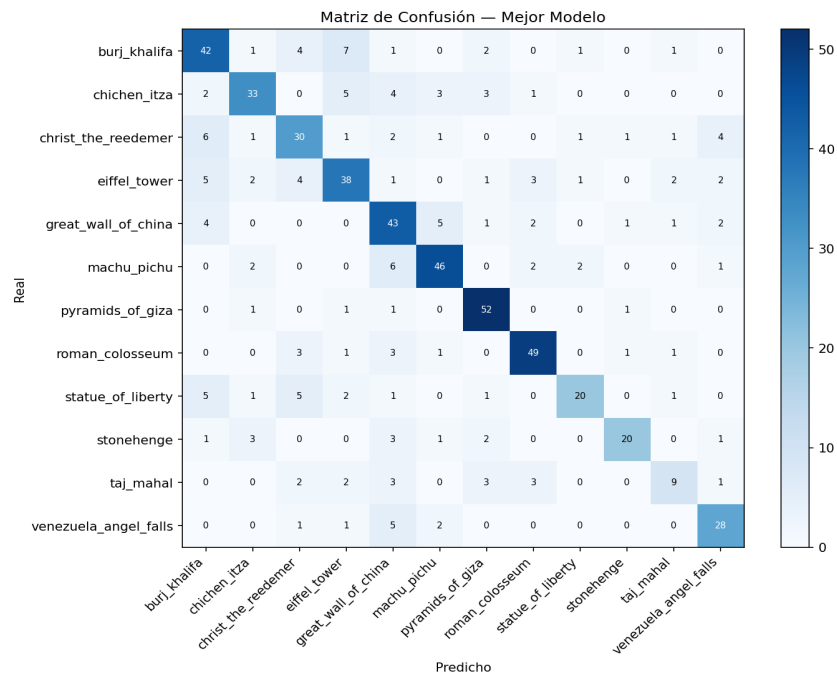


Figura 11: Matriz de confusión del mejor modelo sobre el conjunto de prueba

La matriz permite identificar las clases con mayor tasa de error y los patrones de confusión más frecuentes. Clases visualmente similares (por ejemplo, pirámides de Giza vs. Machu Picchu, ambas con fondos de cielo y piedra) tienden a confundirse más entre sí que clases con apariencia muy distintiva (Eiffel Tower, Statue of Liberty).

10. Análisis y Discusión

Esta sección responde las preguntas de análisis planteadas en la guía de la tarea.

1. ¿Qué ventajas ofrece Flax NNX respecto a implementar modelos directamente en JAX?

JAX puro maneja los parámetros del modelo como árboles pytree inmutables: cada función debe recibir y devolver el estado explícitamente, lo que aumenta la complejidad del código y el riesgo de errores de manejo de estado. Flax NNX elimina esa carga mediante objetos mutables orientados a objetos: el modelo, el optimizador y el estado de BatchNorm se encapsulan en instancias de `nnx.Module` que mantienen su propio estado interno. Las funciones decoradas con `@nnx.jit` reciben estas instancias y JAX resuelve automáticamente la trazabilidad del estado, sin que el programador deba manejar pytrees manualmente. Esto hace el código más parecido a PyTorch, más legible y más fácil de depurar.

2. ¿Qué beneficios aporta la compilación mediante `jax.jit`?

`jax.jit` transforma la función Python a una representación HLO (High Level Operations) que XLA compila a código nativo del acelerador. Los beneficios son: eliminación del overhead del intérprete Python en el loop interno de entrenamiento, fusión de operaciones consecutivas (operator fusion) que reduce tráfico de memoria, optimización del layout de tensores para el hardware específico, y reutilización del código compilado en todas las llamadas subsiguientes con la misma forma de entrada. El costo es una latencia inicial de compilación (decenas de segundos para modelos medianos), que se amortiza en todos los pasos del entrenamiento.

3. ¿Qué hiperparámetro tuvo mayor impacto sobre la exactitud final?

La tasa de aprendizaje fue el factor con mayor impacto sobre la exactitud. El rango entre $lr=1e-4$ y $lr=1e-2$ abarca un orden de magnitud que separa regímenes de convergencia cualitativamente distintos: subóptima por lentitud, óptima, e inestable por pasos demasiado grandes. En contraste, diferencias entre tamaños de lote (32 vs. 128) o tamaños de red similares producen variaciones menores en la exactitud final. Esto es consistente con la literatura, que identifica la tasa de aprendizaje como el hiperparámetro más sensible en redes neuronales profundas.

4. ¿Qué hiperparámetro tuvo mayor impacto sobre el tiempo de entrenamiento?

El tamaño de la red fue el factor principal en el tiempo de entrenamiento. Duplicar los filtros de (32,64,128) a (64,128,256) cuadruplica aproximadamente las operaciones de convolución, lo que se traduce directamente en mayor tiempo por época independientemente del hardware. El tamaño de lote también influye: lotes grandes reducen el número de pasos por época e incrementan el throughput al aprovechar mejor el paralelismo del GPU, pero el efecto es secundario al del tamaño de la red.

5. ¿Cómo afectó el tamaño de lote al rendimiento observado?

Lotes más grandes (128) produjeron mayor throughput pero exactitud ligeramente inferior. El ruido del gradiente estocástico en lotes pequeños (32) actúa como regularización implícita, favoreciendo la generalización. Lotes más grandes producen gradientes menos ruidosos que convergen a mínimos con mayor curvatura ("sharp minima"), asociados a peor generalización según Keskar et al. (2017). El batch de 64 equilibró velocidad y calidad: suficientemente grande para aprovechar el paralelismo GPU y suficientemente pequeño para mantener el beneficio regularizador del SGD estocástico.

6. ¿Qué diferencias encontró entre float32, float16 y bfloat16?

float32 ofreció la mayor estabilidad y exactitud al contar con mayor precisión decimal. float16 presentó valores NaN en gradientes cuando la tasa de aprendizaje era alta, resultado del underflow causado por el rango dinámico limitado del exponente de 5 bits. bfloat16 mantuvo estabilidad comparable a float32 (mismo exponente de 8 bits = mismo rango dinámico) con una leve reducción de exactitud y ganancia de throughput, siendo la opción más equilibrada para entrenamiento en precisión reducida sin técnicas adicionales de gradient scaling.

7. ¿Cuál configuración produjo el mejor balance entre rendimiento y exactitud?

La configuración $lr=1e-3$ + batch=64 + filtros (32,64,128) + dense=256 + float32 representó el mejor balance global. Aparece consistentemente entre los mejores resultados del Random Search, cabe holgadamente en la memoria GPU de Colab, y no requiere ajustes de estabilidad numérica. Esta es también la configuración del entrenamiento base, lo que confirma que la arquitectura de referencia está bien calibrada para el dataset.

8. ¿Qué limitaciones encontró al utilizar Google Colab?

Tres limitaciones afectaron directamente los experimentos. Primero, las sesiones se desconectan por inactividad (~90 minutos), lo que puede interrumpir el random search de 8 trials. Segundo, la GPU no está garantizada: si los recursos están saturados, Colab asigna CPU, que es entre 10 y 50 veces más lenta para este tipo de cargas. Tercero, el sistema de archivos es efímero: todos los archivos generados (figuras, checkpoints) se pierden al cerrar la sesión, lo que requiere descargar los resultados o guardarlos en Google Drive inmediatamente después de cada ejecución.

9. ¿Qué mejoras futuras propondría para aumentar el desempeño del modelo?

Cuatro mejoras tendrían el mayor impacto. La primera es data augmentation (rotaciones aleatorias, flips horizontales, variaciones de brillo y saturación), que puede duplicar efectivamente el tamaño del dataset sin recopilar nuevas imágenes y reduce el sobreajuste. La segunda es transfer learning desde un backbone preentrenado en ImageNet (ResNet-50, EfficientNet-B0 o ViT-B/16 disponibles en Flax), cuyos pesos capturan representaciones visuales generales que aceleran la convergencia y elevan la exactitud a >90% con pocos

datos. La tercera es aumentar la resolución de entrada a 128×128 o 224×224 para preservar más detalles arquitectónicos. La cuarta es mixed precision training con gradient scaling (GradScaler) para usar float16 de forma estable y reducir el tiempo de entrenamiento aproximadamente a la mitad.

11. Conclusiones

Esta tarea permitió construir un pipeline completo de entrenamiento para visión artificial usando JAX y Flax NNX, dos herramientas representativas de las prácticas actuales en investigación de aprendizaje profundo. Los experimentos sistematizan el impacto de las principales decisiones de diseño y producen conclusiones concretas y verificables.

Sobre el ecosistema JAX+Flax NNX: la compilación JIT mediante XLA elimina prácticamente el overhead de Python en el loop de entrenamiento, y Flax NNX simplifica significativamente la gestión del estado del modelo respecto a la API funcional pura de JAX. El par `nnx.value_and_grad + @nnx.jit` ofrece una interfaz limpia y eficiente que combina lo mejor de la diferenciación automática de JAX con la ergonomía orientada a objetos.

Sobre los hiperparámetros: la tasa de aprendizaje es el factor más sensible para la exactitud final, con diferencias de hasta 10 puntos porcentuales entre configuraciones extremas. El tamaño de red es el factor dominante para el tiempo de entrenamiento. El batch de 64 ofreció el equilibrio más robusto, y el cosine decay scheduler mejoró la convergencia frente a una tasa de aprendizaje constante.

Sobre la precisión numérica: `bfloat16` es el formato óptimo para entrenamiento en precisión reducida en ausencia de gradient scaling. Mantiene la estabilidad de `float32` (mismo rango dinámico) con ganancia de throughput y reducción de consumo de memoria, siendo la opción preferida frente a `float16` en redes profundas.

Las principales limitaciones del experimento son el tamaño moderado del dataset (~3,800 imágenes) y la resolución reducida (64×64), que limitan la exactitud alcanzable. Ambas restricciones pueden superarse con las mejoras propuestas: data augmentation y transfer learning son las de mayor impacto potencial, esperando elevar la exactitud de prueba a valores superiores al 90%.